

SPI Flash Controller (SFC)

Technical Design Documentation

Source Code Reference & Module Description

1. Project Overview

The SPI Flash Controller (SFC) is an RTL subsystem designed to interface with SPI NOR flash memories and transfer their contents to a DDR3 memory controller (DDR3MC). The design supports two external flash devices (EEPROMs) connected over a standard SPI bus, providing read, page program (write), and sector erase operations. Readback data from flash is streamed over an asynchronous FIFO pipeline into DDR3 DRAM for high-bandwidth storage.

The SFC also implements a Reprogram (RP) feature: on command, it reads data from a DDR3 update region and reprograms a corresponding flash sector via a 4-byte-address page program sequence. This makes the subsystem capable of both initial flash-to-DRAM data loading and runtime in-system flash reprogramming.

1.1 Key Characteristics

Parameter	Value	Notes
SFC Clock	200 MHz	clk_sfc (800 MHz * PLL / 4)
SPIM Clock	100 MHz	clk_spim — SPI master domain
SPI SCK Frequency	~6.25 MHz	25 MHz system / 4 divider in sfc_spim_ctrl
Flash Devices Supported	2	A1 and A2 (MAX_EEPROM_COUNT = 2)
EEPROM Page Size	256 bytes	EEPROM_PAGE_SIZE_IN_BYTES
Flash Size	32 MB per device	FLASH_STORAGE_SIZE
DDR3MC Data Bus Width	512 bits	64 bytes per transaction
DDR3MC Address Width	33 bits	DDR3MC_ADDR_WIDTH
Supported Flash Vendors	Micron, Macronix	Auto-detected via RDID
SPI Mode	Single (Standard SPI)	MOSI/MISO, no Quad-SPI
Async FIFOs Used	3	AC, Data, SFC-to-DDR3MC

1.2 Supported Flash Instructions

Instruction	Description
RDID (9Fh)	Read Manufacturer and Device ID
WREN (06h)	Write Enable
WRDI (04h)	Write Disable
RDSR (05h)	Read Status Register (WIP, WEL bits)

RDFSR (70h) / RDSCUR (2Bh)	Read Flag/Security Status Register (ERR bits)
RD (03h)	Read Data — 3-byte address
RD4 (13h)	Read Data — 4-byte address
PP (02h)	Page Program — 3-byte address
PP4 (12h)	Page Program — 4-byte address
SE4K4 (21h)	4 KB Sector Erase — 4-byte address
BE (C7h)	Bulk Erase
NOP (FFh)	No Operation

2. System Architecture

The SFC is structured as two clock domains bridged by asynchronous FIFOs. The SFC domain (200 MHz) handles command generation and DDR3MC interfacing. The SPIM domain (100 MHz) handles the physical SPI bus transactions.

2.1 Block Diagram (Textual)



2.2 Clock Domains

Domain	Description
clk_sfc (200 MHz)	Main SFC domain: command generation (sfc_cmd_generator), DDR3MC transport (sfc_ddr3mc_transporter), FIFO write-side for AC and data FIFOs, FIFO read-side for sfc_to_ddr3mc FIFO
clk_spim (100 MHz)	SPI master domain: sfc_spim_top, sfc_spim, sfc_spim_ctrl, sfc_spim_read/write/erase_control — all physical SPI operations

2.3 FIFO Pipeline

FIFO Instance	Type	Description
spim_ac_fifo	fifo_async (std)	Address+Command FIFO. Write: clk_sfc (cmd_generator). Read: clk_spim (spim_top). Width = CMD(6) + ADDR(32) = 38 bits. Depth = 4.

spim_data_fifo	fifo_async_piso	Write data FIFO. Parallel-In Serial-Out. Writes one 256-byte page (2048 bits) at a time from clk_sfc. Reads 8 bits/cycle in clk_spim. Depth = 4.
sfc_to_ddr3mc_fifo	fifo_async_sipo	Read data FIFO. Serial-In Parallel-Out. Writes 8 bits/cycle from clk_spim. Reads 512 bits (64 bytes) at a time in clk_sfc. Depth = 4.

3. Source File Descriptions

This section documents every source file in the SFC project, organized by folder.

include/

sfc_parameters.vh

Global parameter header included by all RTL and simulation files in the SFC project. Contains all shared constants, enumerated values, and derived parameters.

Key Parameter Groups

Group	Description
Clock Periods	SFC_CLK_PERIOD_ns = 1.25 ns (800 MHz), SPIM_CLK_PERIOD_ns = 10 ns (100 MHz)
EEPROM/Flash Sizing	EEPROM_STORAGE_SIZE_IN_BYTES = 128 MB (sim: 128 B), PAGE_SIZE = 256 B, MAX_EEPROM_COUNT = 2, FLASH_STORAGE_SIZE = 32 MB
AC Timing	tRLRH_ns = 30,000 ns (reset-low hold time), tRHSL_ns = 10,000 ns (reset-high setup). Converted to clock counts for hardware timing.
DDR3MC Interface	CMD_WIDTH = 4, ADDR_WIDTH = 33, DATA_WIDTH = 512, NOP/WRITE/READ command encodings, start/end addresses DDR3MC_SA/EA, update region addresses
Status Bits	STATUS_BIT_CMD_ERR [0], ADDR_ERR [1], PROGRAM_ERR [2], ERASE_ERR [3], DDR3MC_TRANSPORT_DONE [4], READY_TO_REPROGRAM [5]
Flash Array Map	16 flash array IDs (A1..D4), each mapped to a contiguous address range of EEPROM_STORAGE_SIZE_IN_BYTES
Flash Instructions	All SPI opcodes: WREN, WRDI, RDSR, RDFSR, RD, RD4, PP, PP4, SE4K4, BE, DP, RES, RDID, RDSCUR
SPIM_CTRL Registers	3-bit register map: DATA_REG(0), CTRL_REG(1), ADDR1_REG(2)..ADDR4_REG(5). Status bits: BUSY[0], TxEMPTY[1], RxREADY[2], ISWAIT6[3]
Erase/Write/Read Modes	Enumerated mode codes for erase (CE, SE4K, SE4K4, SE32K, SE64K, SE64K4) and write (PP, PP4) and read (RD, RD4)

FIFO Parameters	AC FIFO: 38-bit wide, depth 4. Data FIFO: 2048-bit write / 8-bit read, depth 4. SFC-DDR3MC FIFO: 8-bit write / 512-bit read, depth 4.
Flash Manufacturer IDs	FLASH_MANUID_MACRONIX = 8'hC2, FLASH_MANUID_MICRON = 8'h20 — used to select the correct error status register to poll

rtl/

sfc_wrap.sv

Top-level FPGA integration wrapper. Instantiates the PLL/MMCM clock generators, reset synchronizer, sfc_top, the test driver, and the DDR3MC dummy model. This module is the chip boundary — it connects to external FPGA pins or, in simulation, to the sfc_bfm bus functional model.

Sub-instances

Instance	Description
sfc_clkgen	MMCM generating clk_1 (200 MHz) from differential input gclk_p/gclk_n
tmb_clkgen	MMCM generating 100 MHz from CLK_G4P/N for the TMB interface
reset_sync	Two-flop reset synchronizer for clk_1 domain
sfc_top	Main SFC functional core
sfc_test_driver	Simulation/VIO test stimulus generator
ddr3mc_dummy	Behavioral DDR3 memory model for simulation (BRAM-backed) or TMB board interface for hardware testing

Compile-time Switch

SYNTHS: when defined, exposes FPGA pins (gclk, greset_n, spi_sck[A..D], etc.) as actual ports and connects to the TMB hardware interface. When undefined (simulation), drives clocks and reset internally and connects sfc_bfm.

sfc_top.v

Functional integration top. Wires together all SFC sub-blocks with their interconnecting FIFOs. Does not contain any RTL logic itself beyond signal routing and output assignments. This is the level at which the SFC is instantiated in the MemCtrl1 system.

Port Summary

Port	Dir	Width	Description
clk_sfc	in	1	200 MHz SFC domain clock
clk_sfc_reset_n	in	1	Active-low reset, SFC domain

clk_spim	in	1	100 MHz SPIM domain clock
clk_spim_reset_n	in	1	Active-low reset, SPIM domain
ddr3mc_cmd	out	4	Command to DDR3MC: NOP/READ/WRITE
ddr3mc_addr	out	33	33-bit byte address to DDR3MC
ddr3mc_write_data	out	512	Write data to DDR3MC
ddr3mc_read_data	in	512	Read data returned by DDR3MC
ddr3mc_read_data_valid	in	1	Qualifies ddr3mc_read_data
ddr3mc_ready	in	1	DDR3MC ready to accept command
spi_sck_io	inout	1	SPI clock (to/from flash)
spi_mosi_dq0_io	inout	1	SPI MOSI (data out to flash)
spi_miso_dq1_io	inout	1	SPI MISO (data in from flash)
spi_wpn_dq2_io	inout	1	SPI write-protect (active-low)
spi_holdn_dq3_io	inout	1	SPI hold (active-low)
spi_csn_io	inout	2	Chip selects for 2 flash devices
spi_resetn_io	inout	1	Flash hardware reset (active-low)
status	out	8	Status flags (errors, transport done)
rp_en	in	1	Reprogram enable: triggers RP sequence
rp_sa	in	33	Reprogram start address (DDR3 region)
rp_ea	in	33	Reprogram end address (DDR3 region)
rp_done	out	1	Reprogram complete pulse

Key Internal Connections

SPI outputs (MOSI, WPN, HOLDN, CSN, RESETN, SCK) are driven by sfc_spim_top through sfc_io. MISO is looped from sfc_io input back to sfc_spim_top. All FIFO instances bridge the two clock domains: spim_ac_fifo (cmd+addr), spim_data_fifo (write data, PISO), and sfc_to_ddr3mc_fifo (read data, SIPO).

sfc_io.v

Bidirectional pad ring for all SPI signals. Each SPI pin is implemented using PDUW08DGZ_H_G — a standard-cell bidirectional I/O pad primitive with tri-state control. The output-enable (OEN) is active-low, derived from the oe signals. For simulation the pads are transparent (wire-through behavior).

SPI Signals and Their Direction

Signal	Direction & Notes
spi_sck	Output only. oe = 1'b1 always (SFC is SPI master, SCK is always output)
spi_mosi_dq0	Output only. oe = 1'b1. MOSI is master-driven in standard SPI mode

spi_miso_dq1	Input only. oe = 1'b0. MISO is input from flash. Comment in sfc_top confirms single-mode SPI only
spi_wpn_dq2	Output only. oe = 1'b1. Write-protect signal driven by controller
spi_holdn_dq3	Output only. oe = 1'b1. Hold signal driven by controller
spi_csn[1:0]	Output only. oe = 2'b11. One chip-select per flash device
spi_resetrn	Output only. oe = 1'b1. Hardware reset to flash devices

sfc_cmd_generator.v

Command generator FSM running in the clk_sfc domain. It sequences flash read and write operations, pushing command+address entries into the spim_ac_fifo and optionally data into the spim_data_fifo for write operations.

State Machine States

State	Description
IDLE	Waits for ddr3mc_ready asserted before proceeding
READY_TO_READ	Waits for spim_ac_fifo to have space; then issues RD4 commands
CMD_READ	Pushes a RD4 command+address into spim_ac_fifo for each flash device (A1, A2)
CMD_READ_WAIT	Counts read commands issued per eeprom_cnt; transitions to READY_TO_REPROGRAM after all EEPROMs read
READY_TO_REPROGRAM	Waits for reprogram commands (PP4 or SE4K4) from sfc_ddr3mc_transporter via cmdgen_cmd interface; signals cmdgen_ready when accepted
MOVE_TO_SPIM_FIFO	Latches the command/address into spim_ac_fifo and (if PP4) data into spim_data_fifo
DONE / FLUSH_WAIT	Transition/cleanup states (FLUSH functionality is disabled/zeroed out in current code)

External Interfaces

Interface	Description
cmdgen_cmd / cmdgen_addr / cmdgen_din	Input from sfc_ddr3mc_transporter. Provides the reprogram command type, target flash address, and 256-byte page data
cmdgen_ready	Output to transporter. Pulses high when the cmd_generator has accepted and forwarded the command to the FIFOs
rp_done	Input from transporter. Used to trigger debug read-back sequence (READ_DEBUG mode)
spim_ac_fifo_wr_*	Write port to spim_ac_fifo (command+address entries)
spim_data_fifo_wr_*	Write port to spim_data_fifo (256-byte write payload for PP4)

sfc_spim_top.v

SPI master top-level dispatcher, running in the clk_spim domain. Pulls command+address entries from spim_ac_fifo, decodes the target flash device address, drives the corresponding chip-select, and dispatches to the sfc_spim engine.

Reset Sequencing

On power-up (or any reset), the FSM starts in APPLY_RESET, holds spi_resetsn low for tRLRH_nCK cycles, then enters RESET_RECOVERY for tRHSL_nCK cycles before going IDLE. This ensures the flash devices complete their hardware reset sequence before any commands are issued.

Flash Device Selection

Command	Address Decoding
RD4 (read)	spim_addr == A1_ID (0) or A2_ID (1)
PP4 / SE4K4 (write/erase)	Address falls in A1_SA..A1_EA or A2_SA..A2_EA
Invalid command	None

Key Outputs

spim_page_addr_err and spim_cmd_err are sticky status bits set on address or command decode errors. spi_resetsn, spi_csn, spi_mosi_dq0 are all registered outputs driven by the state machine.

sfc_spim.v

SPI operation dispatcher. Decodes the 6-bit command field (bits [5:3] = operation type: 001=erase, 010=write, 100=read; bits [2:0] = mode) and activates exactly one of three sub-controllers: sfc_spim_read_control, sfc_spim_write_control, or sfc_spim_erase_control. All three controllers share access to one sfc_spim_ctrl (SPI master) through a mux.

Command Encoding (6-bit cmd field)

cmd[5:3]	cmd[2:0]	Operation
3'b100	READ_CTRL_MODE_RD (001)	CMD_SPIM_RD — 3-byte address read
3'b100	READ_CTRL_MODE_RD4 (101)	CMD_SPIM_RD4 — 4-byte address read
3'b010	WRITE_CTRL_MODE_PP (001)	CMD_SPIM_PP — 3-byte address page program
3'b010	WRITE_CTRL_MODE_PP4 (101)	CMD_SPIM_PP4 — 4-byte address page program

3'b001	ERASE_CTRL_MODE_CE (000)	CMD_SPIM_CE — chip erase
3'b001	ERASE_CTRL_MODE_SE4K4 (101)	CMD_SPIM_SE4K4 — 4KB sector erase, 4-byte addr
3'b001	Other erase modes	SE4K, SE32K, SE64K, SE64K4

Shared SPI Master Mux

sel_master, wr_master, addr_master, d_in_master, and d_out_master are muxed between the three controllers. Only one controller is active at a time (based on read_en/write_en/erase_en). The spi_wp_n signal is asserted during writes and erases. spi_hold_n is tied high (no hold used).

sfc_spim_ctrl.v

Low-level SPI master controller. Implements the physical SPI protocol — serializing outgoing bytes (command, address bytes, data) and deserializing incoming bytes (status registers, read data) over the SCK/MOSI/MISO lines. Operates with a 4x clock divider (25 MHz system -> 6.25 MHz SPI).

Register Interface (3-bit addr, 8-bit data)

addr[2:0]	Operation
3'b000 + wr=0	Read rx_data (received byte from flash)
3'b001 + wr=0	Read status: bit0=busy, bit1=tx_empty, bit2=rx_ready, bit3=is_wait6
3'b001 + wr=1	Write command byte (triggers SPI transaction)
3'b010 / 011 / 100 / 101 + wr=1	Write address byte 1..4 (b1=LSB, b4=MSB)
3'b000 + wr=1	Write TX data byte

FSM States

IDLE -> TxCMD -> [WAIT1] -> TxADD_B4/B3/B2/B1 -> [WAIT] -> TxDATA / TxDUMMY / RxDATA -> CLR_CMD -> IDLE. The WAIT states insert one-cycle pipeline delays between address/data phase transitions. WAIT6 stalls during PP/PP4 data streaming, polling tx_new_data.

Clock Generation

An internal divider generates spi_clk_int at $\text{SYS_FREQ/SPI_FREQ} = 25 \text{ MHz} / 6.25 \text{ MHz} = 4$. The SPI clock is held high in idle states and toggled only during tx_enable or rx_enable phases. clk_en pulses once per SPI period to advance the state machine.

sfc_spim_read_control.v

FSM that drives the sfc_spim_ctrl register interface to execute a complete flash read sequence: issue the read instruction (RD or RD4), send address bytes, then receive and stream all bytes from the flash page to the sfc_to_ddr3mc_fifo.

Read Sequence (RD4 path)

Phase	Action
SEND_INSTR_RD4	Write FLASH_INSTR_RD4 (0x13) to sfc_spim_ctrl CMD register
CHECK_BUSY_1	Poll ctrl reg; wait for busy=1 (transaction started)
SEND_ADDR_B4..B1	Write 4 address bytes (MSB first) to ADDR4..ADDR1 registers
CHECK_RXREADY_1	Poll for rx_ready; flash data is streaming in
SAMPLE_DATA_BYTE	Read received byte from DATA register
CHECK_READY	Backpressure: wait for downstream FIFO (sfc_to_ddr3mc_fifo) to be ready
CHECK_RXREADY_0	If more bytes remain (byte_cnt > 1): wait for rx_ready to deassert, then loop
CHECK_BUSY_0	After all bytes received: wait for busy=0 (transaction complete)
DONE	Signal done; return to IDLE when en deasserts

Reads EEPROM_STORAGE_SIZE_IN_BYTES bytes per invocation (128 MB in hardware, 128 bytes in simulation). valid/data signals carry each received byte downstream.

sfc_spim_write_control.v

FSM that executes a complete flash page program sequence (PP or PP4), including the required Write Enable (WREN) preamble and post-write status polling.

Write Sequence (PP4 path)

Phase	Action
SEND_INSTR_WREN	Issue WREN (0x06) to enable writes
CHECK_BUSY_0_WREN	Wait for WREN transaction to complete
SEND_INSTR_PP4	Issue PP4 (0x12) page program command
SEND_ADDR_B4..B1	Send 4 target address bytes
CHECK_TXEMPTY/VALID	Handshake with spim_data_fifo: wait for valid data, then write byte
SEND_DATA_BYTE	Write one data byte to ctrl DATA register; assert ready to advance fifo
NEXT_DATA_BYTE	Decrement byte_cnt; loop for all 256 bytes of the page
SEND_INSTR_NOP	Terminate transaction with NOP after last byte
SEND_INSTR_RDSR	Read Status Register to check WIP (Write In Progress) bit
STORE_STATUS_RDSR	Latch WIP and WEL from RDSR response
SEND_INSTR_RDFSR	Read Flag Status Register to check PERR (program error) bit
COMPARE	If WIP or WEL still set, loop back to poll RDSR; else proceed to DONE
DONE	Completed; err output reflects ERR flag

sfc_spim_erase_control.v

FSM that executes a 4KB sector erase (SE4K4) operation. Before erasing, it first reads the flash JEDEC ID (RDID) to identify the manufacturer, which determines which error status register to poll after erasure (RDFSR for Micron, RDSCUR for Macronix).

Erase Sequence

Phase	Action
SEND_INSTR_RDID	Issue RDID (0x9F) to read Manufacturer ID + Device ID (3 bytes total)
STORE_RDID	Latch ManuID and DevID[15:0]; counter cnt counts 2 down to 0
SEND_INSTR_WREN	Issue Write Enable
SEND_INSTR_SE4K4	Issue 4KB sector erase with 4-byte address (0x21)
SEND_ADDR_B4..B1	Send target sector address
SEND_INSTR_RDSR	Poll status register for WIP bit
SEND_INSTR_RDFSR	Poll error status: RDSCUR (0x2B) for Macronix, RDFSR (0x70) for Micron
COMPARE	If WIP or WEL still set: re-poll; else DONE
DONE	err reflects erase error flag

Note: Bulk Erase (CE), 3-byte sector erases, and other modes are defined in the parameters but only SE4K4 is the active path in the current code (IDLE state case statement).

sfc_ddr3mc_transporter.v

FSM running in the clk_sfc domain that manages both the initial flash-to-DDR3 data transport and the reprogram (RP) flow. It is the central orchestrator connecting the DDR3MC memory interface with the cmdgen and flash read data paths.

Phase 1: Initial Transport (Flash -> DDR3)

State	Description
TRANSFER	Monitor sfc_to_ddr3mc_fifo; when data is available and DDR3MC is ready, proceed
PULL_DATA	Assert fifo read-enable to fetch 512-bit (64-byte) chunk
PULL_DATA_WAIT	Pipeline wait for fifo output to stabilize
CMD	Latch fifo data into ddr3mc_write_data
NEXT	Issue DDR3MC WRITE command; increment address by 8 (byte units); loop until DDR3MC_EA reached
DONE	All flash data has been transferred to DDR3; transport_done asserted

Phase 2: Reprogram (DDR3 -> Flash)

State	Description
RP_EN	Wait for rp_en from test driver / external trigger
RP_CMDGEN_SE4K4_WAIT/SE4K4	Send SE4K4 erase command to cmd_generator for the target sector; wait for cmdgen_ready handshake
RP_DDR3MC_READY/READ	Issue up to 4 READ commands to DDR3MC to fill 4 x 512 bits = 2048 bits = 256 bytes of page data
RP_READ_VALID_CNT	Count incoming ddr3mc_read_data_valid pulses (expect 4); accumulate data into cmdgen_din
RP_CMDGEN_READY/PP4	Send PP4 page program command with accumulated data to cmd_generator
RP_NEXT_PAGE	Advance address; loop until rp_ea is reached
RP_DONE	All pages reprogrammed; rp_done pulsed; wait for rp_en to deassert

cmdgen_din is assembled by shifting in DDR3MC read data: {ddr3mc_read_data, cmdgen_din[2047:512]} — MSB-first concatenation of 4 back-to-back 512-bit reads.

sfc_test_driver.sv

Simulation and hardware test stimulus module. In simulation (ifndef SYNTHS), uses an initial block to apply test sequences: after 30 us from reset, enables rp_en with a target address range covering a 4 KB region in the DDR3 update area, then drives a second reprogram operation to the next 4 KB region. In hardware (SYNTHS defined), exposes a Xilinx VIO (Virtual I/O) probe to allow interactive control via Vivado Hardware Manager.

State Machine

State	Description
IDLE	Default state; waits for test_en to be asserted
EN	Asserts rp_en; waits for rp_done to confirm reprogram completed
DIS	Deasserts rp_en for one cycle
DONE	Returns to IDLE for next test iteration

ddr3mc_dummy.sv

Behavioral DDR3 memory controller surrogate used during SFC standalone simulation and FPGA hardware testing. In simulation, backs the memory with a Verilog array initialized from memory.mem. In hardware (SYNTHS), instantiates a TMB (Test Mother Board) interface with real BRAM.

Functional Behavior

Signal	Behavior
ddr3mc_ready	Tied to 1 — always ready (no latency modeled)
ddr3mc_read_data_valid	Asserted on the cycle after a READ command is issued
ddr3mc_read_data	Returns {bram_read_data, bram_read_data} — 256-bit BRAM data duplicated to 512 bits
bram_address	Derived as ddr3mc_addr >> 3 (byte address to word address, since BRAM data is 256 bits = 32 bytes wide)
bram_write_enable	Asserted on WRITE commands; writes ddr3mc_write_data to bram_address

reset_sync.v

Standard two-flop reset synchronizer for metastability-safe reset deassertion. Used in sfc_wrap to synchronize the external FPGA reset (greset_n AND locked) into the clk_1 domain. Parameterized by RESET_O_ACT_LVL (default active-low). Reset assertion is asynchronous; deassertion is registered through two flip-flops to prevent metastability.

import/ — Async FIFO Library

The SFC project imports three async FIFO variants, all sharing the same gray-code pointer synchronization infrastructure. Each depends on the `code_converters` submodule.

import/FIFO_async/rtl/fifo_async.v

Standard asynchronous FIFO with independent write (`wrclk`) and read (`rdclk`) clocks. Configurable by `FIFO_WIDTH` and `FIFO_PTR` (address depth = $2^{\text{FIFO_PTR}}$). Provides full/empty flags, `room_avail` and `data_avail` counts. Uses gray-code pointer synchronization to safely cross the clock domain boundary.

- Used in `sfc_top` as `spim_ac_fifo` (CMD+ADDR FIFO, 38 bits wide, depth 4)
- Supports pointer reset (`reset_wrptr` / `reset_rdptr`) for flush capability

import/FIFO_async_PISO/rtl/fifo_async_piso.v

Asynchronous FIFO with Parallel-In Serial-Out conversion. Accepts wide write data (`WR_DATA_W` bits, e.g. 2048 bits = 256 bytes) and outputs narrower read data (`RD_DATA_W` bits, e.g. 8 bits). Internally stores full-width entries and serializes them byte-by-byte on the read side.

- Used in `sfc_top` as `spim_data_fifo`
- Write side (`clk_sfc`): receives a complete 256-byte flash page in a single write
- Read side (`clk_spim`): provides one byte per read — feeds into `sfc_spim` write datapath

import/FIFO_async_SIPO/rtl/fifo_async_sipo.v

Asynchronous FIFO with Serial-In Parallel-Out conversion. Accepts narrow write data (`WR_DATA_W` bits, e.g. 8 bits) and outputs wide read data (`RD_DATA_W` bits, e.g. 512 bits). Assembles individual bytes into a wide word before signaling read-ready.

- Used in `sfc_top` as `sfc_to_ddr3mc_fifo`
- Write side (`clk_spim`): accepts one received byte at a time from `sfc_spim_read_control`
- Read side (`clk_sfc`): delivers 512-bit (64-byte) chunks to `sfc_ddr3mc_transporter` for DDR3 write

import/FIFO_async/import/code_converters/rtl/binary_to_gray.v

Combinatorial binary-to-Gray code converter. Used internally by all three async FIFOs to convert write and read pointer binary counts into Gray code before synchronization across clock domains. Gray code guarantees only one bit changes per count transition, eliminating multi-bit metastability during pointer synchronization.

import/FIFO_async/import/code_converters/rtl/gray_to_binary.v

Combinatorial Gray-to-binary converter. Used in async FIFOs to convert the synchronized (opposite-domain) gray-code pointer back to binary for comparison in full/empty flag generation. Shared by all three FIFO variants via the `code_converters` submodule.

sim/ — Testbench & BFM Files

[sim/sfc_bfm.sv](#)

SPI Bus Functional Model (BFM) for SFC simulation. Instantiated in sfc_wrap (sim mode). Connects to all four SPI channels (A..D) and models the behavior of flash devices on the bus — providing responses to RDID, RDSR, read, write, and erase SPI transactions for verification of the SFC design. Primarily used to close the SPI loop without real flash hardware.

[sim/tb_sfc_top.sv](#)

Top-level testbench for sfc_top. Instantiates sfc_top directly with driven clock/reset and connects BFM-like stimulus. Verifies the full SFC pipeline from command generation through FIFO transfer to DDR3MC output.

[sim/tb_spim_top.sv](#)

Unit testbench for sfc_spim_top. Tests the SPI master top module in isolation — drives spim_ac_fifo and spim_data_fifo inputs and observes SPI bus outputs and sfc_to_ddr3mc_fifo writes.

[sim/tb_spim.sv](#)

Unit testbench for sfc_spim. Exercises the read/write/erase dispatcher and verifies correct delegation to the three sub-controllers and the SPI master.

[sim/tb_spim_ctrl.sv](#)

Unit testbench for the sfc_spim_ctrl SPI master controller. Verifies the register-mapped command interface: write cmd, write address bytes, write data, check status, read data. Exercises full SPI transactions at the lowest level.

[sim/tb_erase_control.sv](#)

Unit testbench for sfc_spim_erase_control. Verifies the erase FSM including RDID -> WREN -> SE4K4 -> RDSR polling -> RDFSR polling -> COMPARE -> DONE sequence for both Micron and Macronix flash models.

[sim/tb_readWriteErase.sv](#)

Integration-level testbench exercising a full read-write-erase flow. Verifies end-to-end behavior: read flash content via RD4, reprogram a sector (SE4K4 + PP4), verify via another read.

[sim/memory.mem](#)

Hex memory initialization file. Loaded by ddr3mc_dummy in simulation (\$readmemh). Contains the initial BRAM contents representing the DDR3 update region data used during reprogram operations. Format: one 256-bit (64-hex-digit) hex value per line.

sim/25AA1024.v

Behavioral Verilog model of a 25AA1024 SPI EEPROM device (Microchip). Used in some simulation configurations as the flash device under test. Responds to standard SPI read/write/erase sequences.

sim/Macronix-SPIFlash-1Gb/MX66U1G45G/MX66U1G45G.v

Official Macronix vendor behavioral model for the MX66U1G45G 1 Gb SPI NOR flash memory. Implements the complete Macronix command set including RDSCUR (security register with erase/program error bits). Used in simulation to validate Macronix-specific RDFS handling in sfc_spim_erase_control.

sim/Micron-SPIFlash-1Gb/MT25QU01GBBB8E0_VG13/code/N25Qxxx.v

Official Micron vendor behavioral model for the MT25QU01G 1 Gb QSPI NOR flash. Supports standard SPI and QSPI modes. Used in simulation for Micron-specific RDFS (Flag Status Register) polling validation. Configured via the include/ header files (DevParam.h, TimingData.h, UserData.h) in its package.

const/ — FPGA Timing Constraints

The const/ folder contains Xilinx Design Constraints (XDC) files for FPGA implementation on two target boards: F1A (fpgaA, used for SPI group A) and F4A (fpgaBD, used for SPI groups B/C/D). Files come in x1 (single-rank) and x3 (triple-rank) variants.

File	Purpose
sfc_io_F1A_fpgaA_x1.xdc	I/O pin assignments for SPI bus signals on FPGA A (F1A board, single-rank memory)
sfc_io_F1A_fpgaA_x3.xdc	I/O pin assignments for SPI bus signals on FPGA A (triple-rank memory)
sfc_io_F4A_fpgaBD_x1.xdc	I/O pin assignments for SPI bus signals on FPGA B/D (F4A board, single-rank)
sfc_io_F4A_fpgaBD_x3.xdc	I/O pin assignments for SPI bus signals on FPGA B/D (triple-rank)
sfc_wrap_F1A_fpgaA_x1.xdc	Clock, timing, and other top-level constraints for sfc_wrap on FPGA A, single-rank
sfc_wrap_F1A_fpgaA_x3.xdc	Clock, timing, and other top-level constraints for sfc_wrap on FPGA A, triple-rank
sfc_wrap_F4A_fpgaBD_x1.xdc	Clock, timing, and other top-level constraints for sfc_wrap on FPGA B/D, single-rank
sfc_wrap_F4A_fpgaBD_x3.xdc	Clock, timing, and other top-level constraints for sfc_wrap on FPGA B/D, triple-rank

scripts/ — Build & IP Scripts

File	Purpose
sfc_envsetup.tcl	Environment setup script. Sets project variables, source/sim file lists, and include paths. Sourced first by other scripts to establish the build context.
sfc_define.tcl	Defines Tcl variables for compile-time parameters (e.g. SIM flag, board variant selection). Allows reuse of the same RTL with different parameter overrides.
sfc_ip.tcl	Vivado IP integration script. Creates and manages IP cores used by SFC (e.g. MMCM clock generators sfc_clkgen and tmb_clkgen). Can be sourced in Vivado to regenerate IP.
runvcs	VCS (Synopsys) compile and simulation run script for Linux. Compiles all RTL and sim files with correct include paths and runs the top-level simulation.

4. End-to-End Data Flow

4.1 Flash-to-DDR3 Initial Load

1. sfc_ddr3mc_transporter waits in TRANSFER; sfc_cmd_generator waits in IDLE for ddr3mc_ready.
2. Once DDR3MC is ready, cmd_generator transitions to READY_TO_READ. For each of the 2 EEPROMs, it pushes a CMD_SPIM_RD4 entry (with EEPROM ID as address) into spim_ac_fifo.
3. sfc_spim_top (clk_spim) sees entries in spim_ac_fifo, pulls them, decodes flash device, and starts sfc_spim with en=1, cmd=RD4, addr=0.
4. sfc_spim activates sfc_spim_read_control. It drives sfc_spim_ctrl to issue the RD4 instruction and 4 address bytes, then streams received bytes into sfc_to_ddr3mc_fifo byte by byte.
5. sfc_to_ddr3mc_fifo (SIPO) accumulates 64 received bytes, then presents 512 bits to sfc_ddr3mc_transporter.
6. sfc_ddr3mc_transporter writes each 512-bit chunk to DDR3MC with incrementing address until DDR3MC_EA is reached. transport_done is asserted.

4.2 Reprogram Flow (DDR3 -> Flash)

1. External agent (sfc_test_driver or system logic) asserts rp_en with rp_sa and rp_ea defining the DDR3 region.
2. sfc_ddr3mc_transporter enters RP_EN, advances to send SE4K4 erase command to cmd_generator via cmdgen interface.
3. cmd_generator forwards SE4K4 to spim_ac_fifo; sfc_spim_top dispatches to sfc_spim_erase_control, which executes the full RDID->WREN->SE4K4->poll sequence.
4. After erase confirms, transporter issues 4 sequential DDR3 READ commands to fetch 256 bytes (4 x 64 bytes) into cmdgen_din.
5. Transporter sends PP4 command + 256-byte data to cmd_generator via cmdgen interface.
6. cmd_generator pushes PP4 to spim_ac_fifo and page data to spim_data_fifo; sfc_spim_write_control executes the WREN->PP4->data->poll sequence.
7. Transporter advances cmdgen_addr by 256, moves ddr3mc_addr forward, and repeats until rp_ea is reached. rp_done is pulsed.

5. Module Summary

Module / File	Location	Function
sfc_parameters.vh	include/	Shared parameters: flash instructions, timing, FIFOs, DDR3MC, status bits
sfc_wrap.sv	rtl/	FPGA top wrapper: MMCM clocks, reset sync, test driver, DDR3 dummy
sfc_top.v	rtl/	Functional integration: wires all sub-blocks and FIFOs
sfc_io.v	rtl/	Bidirectional pad ring for SPI signals (PDUW08DGZ_H_G cells)
sfc_cmd_generator.v	rtl/	Sequences read/write commands into FIFOs (clk_sfc domain)
sfc_spim_top.v	rtl/	SPI dispatcher: chip-select demux, reset sequence, dispatches to sfc_spim
sfc_spim.v	rtl/	Operation dispatcher: activates read_ctrl / write_ctrl / erase_ctrl per cmd
sfc_spim_ctrl.v	rtl/	Physical SPI master: serializes/deserializes bytes over SCK/MOSI/MISO
sfc_spim_read_control.v	rtl/	FSM for flash read: RD/RD4 instruction, address, byte streaming
sfc_spim_write_control.v	rtl/	FSM for page program: WREN+PP/PP4, data streaming, WIP polling
sfc_spim_erase_control.v	rtl/	FSM for sector erase: RDID, WREN, SE4K4, error status polling
sfc_ddr3mc_transporter.v	rtl/	Orchestrates Flash->DDR3 load and DDR3->Flash reprogram flows
reset_sync.v	rtl/	Two-flop reset synchronizer for safe deassertion
sfc_test_driver.sv	rtl/	Test stimulus: drives rp_en/sa/ea in sim (initial block) or via VIO
ddr3mc_dummy.sv	rtl/	DDR3MC behavioral model: BRAM-backed memory for simulation
fifo_async.v	import/FIFO_async/rtl/	Standard async FIFO (used for spim_ac_fifo)
fifo_async_piso.v	import/FIFO_async_PISO/rtl/	Parallel-In Serial-Out async FIFO (spim_data_fifo)
fifo_async_sipo.v	import/FIFO_async_SIPO/rtl/	Serial-In Parallel-Out async FIFO (sfc_to_ddr3mc_fifo)
binary_to_gray.v	import/.../code_converters/	Binary to Gray code converter for FIFO pointer sync

gray_to_binary.v	import/.../code_converters/	Gray to binary converter for FIFO full/empty generation
sfc_bfm.sv	sim/	SPI bus BFM: models flash response in simulation
tb_sfc_top.sv	sim/	Testbench for sfc_top integration
tb_spim_top.sv	sim/	Unit testbench for sfc_spim_top
tb_spim.sv	sim/	Unit testbench for sfc_spim dispatcher
tb_spim_ctrl.sv	sim/	Unit testbench for SPI master controller
tb_erase_control.sv	sim/	Unit testbench for erase FSM
tb_readWriteErase.sv	sim/	Integration testbench for read-write-erase flow
memory.mem	sim/	Hex init file for DDR3MC dummy BRAM
25AA1024.v	sim/	Microchip SPI EEPROM behavioral model
MX66U1G45G.v	sim/Macronix/	Macronix 1 Gb SPI flash vendor model
N25Qxxx.v	sim/Micron/	Micron 1 Gb QSPI flash vendor model
sfc_envsetup.tcl	scripts/	Build environment setup
sfc_define.tcl	scripts/	Compile-time parameter definitions
sfc_ip.tcl	scripts/	Vivado IP core generation script
sfc_io*.xdc	const/	I/O pin assignments for F1A and F4A boards
sfc_wrap*.xdc	const/	Timing and clock constraints for synthesis

— End of Document —